

COBOL Modernization Portfolio: Demonstrable Project Ideas with Open Tooling

Evidence base and design constraints

COBOL persists in “mainframe-shaped” problem spaces because the workload patterns remain economically and operationally hard to outcompete: large-scale batch (millions of records, end-of-day / end-of-period runs), high-concurrency online transaction processing (OLTP), and long-lived systems-of-record that prioritize correctness, auditability, recoverability, and stable change control over rapid rewrites. ¹

Mainframe usage is not hypothetical or niche: survey-driven industry commentary continues to describe the mainframe as system-of-record infrastructure for banks, insurance companies, governments, airlines, and retailers, with respondents still viewing it as a long-term platform. ²

Two modernization patterns repeatedly show up in credible vendor guidance because they reduce operational risk in long-lived systems: - **Strangler / façade**: introduce an intercepting façade that keeps the client interface stable while functionality is incrementally replaced. ³

- **Anti-corruption layer**: isolate legacy semantics (formats, protocols, error models) behind a translation/adaptation boundary to prevent “legacy leakage” into new services. ⁴

Your constraint—“**demonstrate locally without a real mainframe**”—is realistic if you choose projects that simulate mainframe realities (record-oriented files, job flows, idempotency, reconciliation, audit logs). **GnuCOBOL** ⁵ is a strong fit: it produces native executables across common OSes and supports compiler workflows that map well to CI/CD (preprocess, translate, compile, link). ⁶

Scope risk that you must manage explicitly: requiring 12 ideas *and* detailed build plans for the top 3 pushes you toward projects with reusable scaffolding (data generator, file parsers, job runner, audit log) so each demo doesn’t become a one-off.

Sectors, persistence drivers, and modernization pain points

Sector selection logic

To keep this grounded, the sector list is limited to domains repeatedly associated with mainframes/COBOL in mainstream industry and vendor documentation, and to domains whose workload patterns align with batch + OLTP + system-of-record pressures. ⁷

Sectors where COBOL footprints remain economically “sticky”

Below are 8 sectors (within your 6–10 target) with the **reason COBOL persists** and **five modernization pain points** per sector. The pain points are written in modernization-team language and intentionally biased toward what can be turned into demoable artifacts.

Sector	Why COBOL persists here	Five common modernization pain points
Banking core (deposits/ledger/payments)	Mix of high-volume OLTP + heavy end-of-day/end-of-period batch; strict audit/regulatory reporting; “always-on” system-of-record behavior. ⁸	(1) Safely exposing legacy functions to digital channels/API without semantic drift; (2) Idempotency/replay handling for payment-like actions; (3) Sparse documentation of business rules embedded in COBOL; (4) Regression risk across batch closures; (5) Coexistence of legacy and migrated data stores. ⁹
Insurance (policy admin/claims)	Large nightly batch runs (rating, renewals, statements) + long-lived logic; high need for correctness and traceability; stable data formats and procedures over decades. ¹⁰	(1) Batch restart/recovery when runs exceed windows; (2) Complex file exchanges with partners; (3) Test data scarcity and weak automated regression; (4) Domain logic spread across many programs/copybooks; (5) Integration to modern workflow/orchestration tools. ¹¹
Government benefits/tax/unemployment	Massive periodic batch + long retention/audit; continuity requirements; conservative change control; staffing constraints/skills gaps. ¹²	(1) Documentation gaps and retirements → knowledge loss; (2) Modern identity/API access around legacy processes; (3) Coexistence between legacy records and new portals; (4) Testing and “semantic equivalence” across refactors; (5) Operational risk of “big bang” change during peak periods. ¹³
Airlines/travel & transportation reservations	Mission-critical transaction systems with unpredictable concurrency; correctness under failure; long-running system-of-record behavior. ¹⁴	(1) API enablement while preserving transaction semantics; (2) Observability gaps in end-to-end flows; (3) Interface proliferation (old + new); (4) Performance regression risk; (5) Data-sharing constraints during migrations. ¹⁵
Retail/wholesale (sales consolidation, inventory finance)	Classic overnight batch reporting and consolidation; high throughput file processing; system-of-record continuity. ¹⁶	(1) File-driven integrations and brittle record layouts; (2) Batch orchestration and SLA management; (3) Testing gaps for end-to-end batch chains; (4) Dependency mapping across jobs and datasets; (5) Data lineage/audit requirements for finance reporting. ¹⁷

Sector	Why COBOL persists here	Five common modernization pain points
Telecom billing (CDRs, rating, invoicing)	Enormous record volumes; batch rating and invoicing pipelines; stable billing correctness requirements. ¹⁸	(1) Reprocessing/re-rating and audit trails; (2) Format translation and validation; (3) Performance tuning without breaking outputs; (4) Regression harnesses for rating logic; (5) API adapters for downstream digital billing. ¹⁵
Healthcare billing & remittance-like processing	High-volume claims/billing pipelines; strict audit; integration-heavy ecosystems; long-lived “billing rules” logic. ¹⁹	(1) Complex interchange formats and normalization; (2) Reconciliation of remittance vs expected; (3) Coexistence of legacy and modern pipeline stages; (4) Correctness testing under edge-case-heavy inputs; (5) Documentation gaps and interdependency sprawl. ⁹
Manufacturing finance (cost accounting, production finance)	Periodic close processes; large batch reporting; high integrity requirements; long-lived internal controls. ²⁰	(1) Batch close window overruns; (2) Auditability of transformations; (3) Data lineage across multiple jobs; (4) Dependency mapping for impact analysis; (5) Integration to modern BI/ETL without breaking controls. ¹¹

Converting pain points into project opportunity themes

Across sectors, the pain points converge into four opportunity themes that are demonstrable off-mainframe:

- 1) **Legacy integration/API enablement:** strangler façades, anti-corruption translators, stable contracts around unstable internals. ²¹
- 2) **Batch modernization & observability:** restartability, step-level metrics, job graph visibility, operational safety nets aligned with how batch actually runs. ¹⁷
- 3) **Testing/verification & regression harnesses:** golden masters, differential testing, and invariants that reduce “semantic equivalence” risk. ⁹
- 4) **Data pipelines / file interchange:** fixed-width records, VSAM-like access patterns, and host-style encodings/field types (including packed decimals) that remain common integration barriers. ²²

Portfolio of 12 project ideas

Grouping matches your four buckets. Each idea is deliberately **not CRUD**; each centers batch, transaction-style behavior, auditability, reconciliation, idempotency, and/or safe refactoring.

Legacy integration and API enablement

Idea A: Strangler façade for a ledger posting engine with idempotency + reversals (banking/payments)

Problem statement: Digital channels need “post payment / reverse payment / balance inquiry” but the core posting logic is embedded in COBOL batch + file semantics; direct rewrites risk semantic drift and

regulatory failures.

Why COBOL is relevant: Ledger posting and reversals map cleanly to COBOL's record-oriented processing and to transaction-style commit/backout thinking borrowed from OLTP systems; you can model "unit of work" boundaries explicitly and preserve legacy-style determinism. ²³

Target persona: Bank operations + modernization platform team integrating channels with a system-of-record.

MVP scope (2–4 weeks): REST façade with idempotency keys; COBOL posting module; reversal module; nightly reconciliation batch that validates ledger totals vs input totals; append-only audit log.

Architecture: COBOL modules (`POSTPAY` , `REVERSE` , `BALINQ` , `RECONCILE`); storage = indexed "account ledger" + sequential audit journal; interface = REST service (Python/Go) calling COBOL executable or shared module; async optional = file-spool queue.

Demo plan (local): Docker + GnuCOBOL; synthetic payments generator; run REST calls; show idempotent replay; run reconcile job; show exception report.

Evaluation metrics: p95 API latency; postings/sec; idempotency correctness (duplicate requests produce identical ledger state); reconciliation mismatch rate; audit log completeness.

Monetization/ROI: Faster integration for new channels reduces time-to-change; reduced migration risk vs big bang changes; operational risk reduction via audit/reconciliation.

Key risks/mitigations:

- Integration correctness (double-posting) → enforce idempotency key + deterministic transaction IDs;
- Semantic equivalence (reversal edge cases) → golden-master tests on synthetic but comprehensive scenarios;
- Data format drift → strict fixed-width/copybook-aligned schemas + schema versioning.

Idea B: Anti-corruption layer for host-style fixed-width interchange (insurance/retail/telecom)

Problem statement: Modern services consume JSON/REST, but legacy ecosystems still exchange fixed-width host files (often with packed decimals and strict field rules); integration defects are common and hard to audit.

Why COBOL is relevant: COBOL naturally expresses record layouts with PIC clauses and performs high-throughput file transforms deterministically—exactly what fixed-width interchange demands. ²⁴

Target persona: Integration architect or claims/billing ops team that must onboard partners without breaking legacy pipelines.

MVP scope (2–4 weeks): File translator + validator; schema registry (versioned copybooks); "rejects" pipeline with reason codes; REST endpoint to validate/translate; CLI mode for batch use.

Architecture: COBOL transform modules (`PARSEHOST` , `VALIDATE` , `EMITJSON` , `EMITHOST`); storage = staging directory + reject files + audit CSV/JSONL; interface = REST + CLI.

Demo plan (local): Docker; generate 3 versions of a fixed-width schema; ingest files; show validation errors; emit translated JSON; round-trip back to fixed-width.

Evaluation metrics: records/sec; validation precision (correctly rejected invalid records); round-trip fidelity; auditability (every reject has reason + offset + raw bytes).

Monetization/ROI: Cuts partner onboarding time; reduces expensive production file defects; de-risks staged strangler migrations by isolating legacy semantics. ²⁵

Key risks/mitigations:

- Data format issues (packed/overpunched) → start with ASCII+COMP-3 simulation and add encodings incrementally;
- "Hidden rules" → implement rule plug-ins + versioned schemas;
- Drift between schemas and code → generate validator tables from copybook-like schema file.

Idea C: REST job-submission façade for COBOL batch chains (modern ops around legacy batch)

Problem statement: Teams need to programmatically run/monitor batch jobs (status, spool, outputs) from modern pipelines, but legacy batch control is opaque and error-prone when bridged ad-hoc.

Why COBOL is relevant: The “work” is still record processing and report generation; the façade modernizes the control plane without rewriting the core batch programs. ²⁶

Target persona: Platform engineering / DevOps enabling batch-to-DevOps integration.

MVP scope (2–4 weeks): REST endpoints for submit/status/output; job IDs; step logs; idempotent submit; basic concurrency limits; retry policy; audit log.

Architecture: COBOL modules = example batch steps (STEP01_READ, STEP02_SORT, STEP03_REPORT); storage = spool folder + SQLite job metadata; interface = REST service orchestrating executables; queue optional = local work queue.

Demo plan (local): Docker Compose; submit a multi-step job; stream logs; simulate failure and restart; export metrics.

Evaluation metrics: job throughput; mean time to detect failure; restart success rate; SLA compliance for synthetic workloads.

Monetization/ROI: Reduces manual operator time; reduces outage duration; creates a safe path to strangler-style modernization by stabilizing interfaces. ²⁷

Key risks/mitigations:

- “Façade becomes bottleneck” → rate limiting + async execution;
- Non-deterministic file dependencies → strict input/output contracts + immutable job runs;
- Incomplete observability → structured logs + run manifests.

Batch modernization and observability

Idea D: Restartable batch framework with checkpoints, idempotent outputs, and audit trails (cross-sector)

Problem statement: Batch chains fail mid-run (bad data, timeouts, resource limits) and reruns create duplicates unless restart semantics are engineered; modernization teams need a safe pattern before any refactor.

Why COBOL is relevant: Batch is a primary workload category; the framework is closest to how existing COBOL jobs already operate (sequential flows, file I/O), making adoption realistic. ²⁰

Target persona: Batch operations lead + modernization engineer responsible for batch-window reliability.

MVP scope (2–4 weeks): Checkpoint file per step; restart at record key; idempotent “write once” output strategy; reject handling; end-of-run reconciliation totals; structured run log.

Architecture: COBOL library (CHKPT-READ/WRITE, AUDIT-WRITE, TOTALS) + sample job programs; storage = checkpoint + output + audit journal; interface = CLI runner with “restart-from” and “dry-run”.

Demo plan (local): 10M-record synthetic dataset; inject failure at Nth record; restart and prove identical outputs; show audit trail and reconciliation.

Evaluation metrics: restart time; duplicate output rate (target 0); records/sec; percentage of failures recoverable without manual edits.

Monetization/ROI: Direct reduction in batch-related incidents and rerun labor; de-risks future refactors by adding safety rails first.

Key risks/mitigations:

- Incorrect checkpoint boundaries → checkpoint per “unit of work” and validate via reconciliation;
- Output idempotency mistakes → write to temp + atomic rename + output manifests;
- Performance overhead → configurable checkpoint frequency.

Idea E: Job-flow dependency graph + runbook generator for “JCL-like” batch chains (documentation/dependency mapping)

Problem statement: Batch environments accumulate hundreds of jobs with implicit dependencies (datasets, step outputs); missing documentation makes modernization slow and risky.

Why COBOL is relevant: COBOL batch steps are the nodes; the project treats COBOL as the executable unit and focuses on mapping dependencies rather than rewriting. Batch chains are a fundamental operating mode in z/OS environments, with sequential flows controlled by job control concepts. ²⁶

Target persona: Release manager / modernization architect doing impact analysis.

MVP scope (2–4 weeks): A small DSL that resembles JCL concepts (JOB/STEP/DD); parser that extracts programs and file dependencies; DAG output (Mermaid/Graphviz); generated runbook (inputs/outputs, restart points).

Architecture: Parser (Python) + graph builder; COBOL sample programs representing steps; storage = YAML job definitions + generated docs; interface = CLI (`depmap build` , `depmap explain JOBX`).

Demo plan (local): Provide 10–20 sample jobs; generate dependency graph; simulate change to one file layout; show impacted jobs automatically.

Evaluation metrics: dependency extraction accuracy; time-to-impact-report; percentage of steps with identified inputs/outputs; “unknown edge” rate.

Monetization/ROI: Cuts discovery time; reduces change risk by making dependencies explicit; accelerates strangler planning by identifying seams. ⁹

Key risks/mitigations:

- False confidence from incomplete data → explicitly label unknowns; require schema annotations;
- DSL mismatch vs real JCL → keep DSL small but map to common concepts (DD-like datasets);
- Large graphs → implement partitioning by domain and subgraphs.

Idea F: Batch observability exporter (structured logs + metrics + anomaly flags) for COBOL runs

Problem statement: Modernization teams often lack step-level runtime telemetry and cannot prove whether changes improved or degraded batch windows.

Why COBOL is relevant: The highest-fidelity insights come from instrumenting at the record-processing and step boundaries that COBOL batch owns. ²⁰

Target persona: SRE/operations supporting batch SLAs and modernization releases.

MVP scope (2–4 weeks): Standard log schema (JSONL); metrics emission (Prometheus textfile or CSV); per-step timers; error taxonomy; run manifest including input hashes and output hashes.

Architecture: COBOL instrumentation module (`LOG-JSONL` , `METRIC-INC` , `TIMER-START/STOP`); storage = log and metrics files; interface = CLI runner that aggregates and prints dashboard-ready summaries.

Demo plan (local): Run two versions of a job with different checkpoint frequencies; compare runtime and error patterns; show anomaly detection on a synthetic “bad day” dataset.

Evaluation metrics: metric completeness; overhead (% runtime); MTTR proxy (time to find failing step); SLA percentile tracking over runs.

Monetization/ROI: Faster diagnosis, fewer overruns, safer change cadence; supports compliance by producing durable run manifests. ⁹

Key risks/mitigations:

- Over-instrumentation overhead → sampling + coarse timers;
- Log volume → rotation + summarization;
- Poor comparability across runs → normalize using input hashes and schema versions.

Testing, verification, and regression harnesses

Idea G: Golden-master regression harness for COBOL batch refactors (correctness harness)

Problem statement: The biggest modernization risk is silent semantic drift—outputs “look plausible” but are wrong; teams need a harness that makes equivalence measurable before any rewrite.

Why COBOL is relevant: You can treat COBOL executables as the reference system and regression-test inputs/outputs at scale, aligning with batch’s deterministic nature. ²⁸

Target persona: Modernization team doing refactors, re-platforming, or compiler migrations.

MVP scope (2–4 weeks): Test case runner; snapshot (“bless”) workflow; canonical diff for fixed-width outputs; record-level and totals-level assertions; coverage proxy via execution paths (where feasible).

Architecture: COBOL program under test + Python harness; storage = fixtures + golden outputs + diff reports; interface = CLI (`gm run`, `gm bless`, `gm diff`, `gm report`).

Demo plan (local): Provide v1 and v2 of a COBOL rating/reconcile program; intentionally introduce a rounding bug in v2; show harness catches it and pinpoints record and field.

Evaluation metrics: defect detection rate; runtime per 1M records; diff precision (field-level vs whole-line); regression suite coverage breadth (input partitions).

Monetization/ROI: Reduces production defect risk; shortens stabilization/testing phases; creates audit evidence of equivalence. ⁹

Key risks/mitigations:

- Golden-master “locks in bugs” → add invariants and targeted property tests;
- Large output diffs → field-aware diff using copybook schema;
- Non-determinism → enforce stable sorting, timestamps pinned in test mode.

Idea H: Differential + property-based testing for calculation engines (correctness harness)

Problem statement: Calculation-heavy domains (rating, billing, tax-like rules) have edge cases that are rarely covered by example-based tests; modernization teams need generative tests that find counterexamples.

Why COBOL is relevant: COBOL modules often encapsulate the canonical rules; differential testing compares COBOL output to a reference function or to invariants, surfacing semantic mismatches early. ²⁹

Target persona: QA lead / engineer modernizing calculation logic from COBOL to another stack.

MVP scope (2–4 weeks): Input generator (property-based); runner that invokes COBOL module; reference implementation for a subset; invariant checks (monotonicity, conservation totals, bounds); shrinking counterexample output.

Architecture: COBOL calc module (`RATE`, `TAX`, or `PRICING`); Python test harness; storage = counterexample corpus; interface = CLI with seed replay for deterministic failures.

Demo plan (local): Run 50k generated cases; show at least one automatically found edge case (e.g., rounding, negative amount, boundary date).

Evaluation metrics: counterexamples found/hour; reproducibility rate; invariant coverage; time-to-minimal-failing-case.

Monetization/ROI: Reduces long-tail defects; increases confidence to decompose systems using strangler increments. ³

Key risks/mitigations:

- Reference implementation mismatch → keep reference scoped and always preserve COBOL as oracle;
- Flaky tests → deterministic seeds and pinned env;
- Input space explosion → partitioning + targeted generators.

Idea I: COBOL auto-documentation + dependency and data I/O inventory (documentation/dependency mapping)

Problem statement: Modernization stalls because teams can't answer basic questions quickly: "Which programs call this?", "Which copybooks define this record?", "Where is this dataset written?", "What is the minimal seam for strangling this feature?"

Why COBOL is relevant: The code is the source of truth; a static analyzer that extracts CALL/COPY/file I/O gives immediate ROI without runtime access to production. ¹³

Target persona: Enterprise architect / modernization discovery team.

MVP scope (2-4 weeks): Parse COBOL sources for PROGRAM-ID, CALL, COPY, SELECT/FD, and key fields; produce call graph + copybook dependency graph; generate Markdown + Mermaid diagrams; export CSV "system inventory."

Architecture: Python analyzer + sample COBOL codebase; storage = generated docs; interface = CLI (docgen scan, docgen graph, docgen impact --symbol X).

Demo plan (local): Bundle a synthetic but realistic codebase (20-40 programs, shared copybooks, batch flow); generate docs; demonstrate change impact on a field definition.

Evaluation metrics: extraction precision/recall on known test fixtures; % of programs fully parsed; time to regenerate docs in CI; "unknown symbol" rate.

Monetization/ROI: Reduces discovery cost; accelerates planning; prevents wrong-cut migrations by exposing true dependencies.

Key risks/mitigations:

- Parser brittleness across dialects → start with patterns that are stable (COPY, CALL, SELECT/FD), then expand;
- False negatives → build a "warnings report" with line numbers;
- Trust → include fixtures and unit tests for the analyzer itself.

Data pipelines and file-based interchange

Idea J: VSAM-like indexed store emulator + change journal (file-based system-of-record simulation)

Problem statement: Many legacy apps depend on keyed access patterns (VSAM-like) and record layouts; modernization teams need a way to prototype migrations and tests without a mainframe.

Why COBOL is relevant: COBOL directly models fixed/variable record definitions and is explicitly used to define record formats for VSAM-like files; emulating keyed access preserves the "shape" of legacy semantics. ³⁰

Target persona: Modernization engineer building off-platform testbeds and migration rehearsals.

MVP scope (2-4 weeks): Create/read/update/delete keyed records; sequential extract (for batch); change journal (append-only); audit of who/what changed; basic consistency checks.

Architecture: COBOL modules (CREATE, READK, UPDATE, DELETE, EXTRACT); storage = indexed file + journal + snapshot export; interface = CLI commands.

Demo plan (local): Load 1M synthetic account/policy records; perform updates; replay journal to rebuild state; demonstrate audit trail.

Evaluation metrics: point lookup latency; batch extract throughput; journal replay correctness; recovery time from journal.

Monetization/ROI: Enables migration practice and regression suites; reduces dependency on scarce mainframe test regions. ³¹

Key risks/mitigations:

- File format mismatch vs real VSAM → position as "semantic emulator" not binary-compatible;
- Concurrency inconsistencies → single-writer mode in MVP; extend later;
- Large file performance → batching and indexed key design.

Idea K: Packed-decimal + encoding-aware fixed-width validation toolkit (format correctness focus)

Problem statement: Interchange failures often come from subtle field encoding issues (packed decimals, sign nibbles, numeric/alphanumeric constraints) that unit tests miss; teams need deterministic validators.

Why COBOL is relevant: COBOL data typing is designed to express these field constraints and can be used as the authoritative validator/parsing engine for host-style records. ²⁴

Target persona: Data engineering + integration QA in file-heavy environments.

MVP scope (2–4 weeks): Schema definition file; validator that outputs errors with byte offsets; “repair” mode for specific fixups; library form (callable subprogram) + CLI wrapper.

Architecture: COBOL validator module; storage = error reports + corrected outputs; interface = CLI + callable subprogram for other COBOL jobs.

Demo plan (local): Generate intentionally corrupted files (bad sign nibbles, overflow, invalid dates); validate; show deterministic, field-level error reasons suitable for ops.

Evaluation metrics: validation throughput; false positive/negative rate against known fixtures; error localization accuracy (byte/field).

Monetization/ROI: Reduces production file rejects and partner escalations; makes migrations safer by catching format drift early.

Key risks/mitigations:

- Overfitting to one schema → strong schema versioning;
- Hard-to-debug binary fields → hex dump in error output;
- Repair mode risk → require explicit, logged transformations.

Idea L: Telecom-style CDR rating + invoicing batch with reconciliation and exception workflow (high-volume batch demo)

Problem statement: Rating pipelines must process huge record volumes, apply deterministic rules, produce invoices, and reconcile totals; modernization teams need a demonstrator that looks like real billing, not CRUD.

Why COBOL is relevant: High-throughput sequential record processing, deterministic transformations, and report-style outputs are core strengths of COBOL batch. ²⁰

Target persona: Billing ops + engineering team modernizing a rating/batch pipeline.

MVP scope (2–4 weeks): Ingest fixed-width CDRs; rate by plan/time band; produce invoice file per customer; reject invalid CDRs; reconcile total rated amount vs aggregate; audit journal of rating decisions.

Architecture: COBOL modules (`CDR-PARSE` , `RATE` , `INVOICE-WRITE` , `REJECTS` , `RECONCILE`); storage = sequential input/output + audit JSONL; interface = CLI runner + optional REST wrapper for “run batch for date.”

Demo plan (local): Generate 5–50M synthetic CDRs; run batch; show invoice generation; show exceptions; show reconcile totals and restart after injected failure.

Evaluation metrics: records/sec; batch completion time; reject rate; reconciliation mismatch rate; restart correctness.

Monetization/ROI: Demonstrates measurable batch cost reduction via performance + restartability; reduces incident risk from reruns; faster rule change cycles with regression harnesses. ⁹

Key risks/mitigations:

- Rule complexity creep → keep MVP rule set small but extensible;
- Performance bottlenecks → streaming I/O and precomputed rate tables;
- Audit volume → sampled audit + full audit on rejects.

Ranking and selection

Scoring uses 1–5 (higher is better). Composite is unweighted sum to keep it transparent.

Idea	Market demand	Demonstrability	Technical depth	Uniqueness	Composite
A Strangler ledger façade	5	5	5	3	18
B Anti-corruption host-file layer	5	4	4	4	17
C REST job-submission façade	4	5	4	4	17
D Restartable batch framework	5	4	4	3	16
E Job-flow dep graph + runbook	4	4	4	4	16
F Batch observability exporter	4	4	3	3	14
G Golden-master regression harness	5	5	4	3	17
H Differential + property testing	4	4	5	4	17
I COBOL auto-doc + dep map	5	5	4	4	18
J VSAM-like indexed emulator	4	4	4	4	16
K Encoding/packed validator toolkit	4	4	3	4	15
L CDR rating + invoicing batch	4	5	4	3	16

Top 3 (highest composite, tie-broken by market demand + portfolio breadth): - **Idea A** (API enablement + transaction-style semantics + reconciliation) - **Idea I** (auto-documentation + dependency mapping; high modernization ROI) - **Idea G** (golden-master regression harness; reduces semantic drift risk)

Detailed build plans for the top three ideas

Build plan for Idea A: Strangler ledger posting façade with idempotency + reversals

Folder structure

```
ledger-modernization/  
  docker/  
    Dockerfile  
    docker-compose.yml  
  cobol/  
    src/  
      POSTPAY.cbl  
      REVERSE.cbl  
      BALINQ.cbl  
      RECONCILE.cbl  
      LIB_AUDIT.cbl  
      LIB_LEDGERIO.cbl  
      LIB_VALIDATION.cbl  
    copybooks/  
      LEDGER-REC.cpy  
      PAYMENT-REQ.cpy  
      AUDIT-REC.cpy  
  api/  
    app.py          # REST façade  
    contracts/  
      openapi.yaml  
    adapters/  
      cobol_runner.py  # subprocess/shared-module adapter  
  data/  
    schemas/  
      ledger_fixedwidth.md  
      payment_request_fixedwidth.md  
    synthetic/  
      gen_payments.py  
      gen_accounts.py  
    samples/  
      payments_2026-02-25.dat  
  tests/  
    unit/  
      test_idempotency.py  
      test_reversal.py  
    golden/  
      cases/  
      expected/  
  scripts/  
    build.sh
```

```
load_demo.sh
run_reconcile.sh
benchmark.sh
README.md
```

Data formats (sample fixed-width schemas)

Payment request input (sequential file or request payload translated to fixed-width before COBOL):

PAYMENT-REQ (length 120)

1-16	TRACE_ID	X(16)	unique idempotency key (external)
17-28	ACCOUNT_ID	X(12)	
29-36	POST_DATE	9(8)	YYYYMMDD
37-52	MERCHANT_REF	X(16)	
53-63	AMOUNT_CENTS	9(11)	implied 2 decimals
64-66	CURRENCY	X(3)	
67-70	CHANNEL	X(4)	WEB POS MOB API
71-72	TXN_TYPE	X(2)	"P" post, "R" reversal request
73-120	FILLER	X(48)	

Ledger record (append-only journal + optional indexed “current balance” store):

LEDGER-REC (length 200)

1-12	ACCOUNT_ID	X(12)	
13-20	POST_DATE	9(8)	
21-36	TRACE_ID	X(16)	idempotency key
37-46	INTERNAL_TXN_ID	X(10)	deterministic hash/sequence
47-57	AMOUNT_CENTS	S9(11)	signed
58-60	CURRENCY	X(3)	
61-62	STATUS	X(2)	OK RV RJ
63-82	REASON_CODE	X(20)	reject/reversal reason
83-102	BAL_BEFORE	S9(11)	
103-122	BAL_AFTER	S9(11)	
123-200	FILLER	X(78)	

Audit log record (JSONL recommended, but also keep fixed-width for “mainframe feel”):

AUDIT-REC (length 160)

1-26	TIMESTAMP_ISO	X(26)	2026-02-25T12:34:56.123Z
27-42	TRACE_ID	X(16)	
43-52	EVENT_TYPE	X(10)	POST REVERSE REJECT RECON
53-57	RESULT	X(5)	OK FAIL
58-160	DETAILS	X(103)	compact message

Module boundaries

- `LIB_VALIDATION` : field validation, range checks, schema version checks, deterministic error codes.
- `LIB_LEDGERIO` : write ledger journal, read indexed current balance, enforce single-writer lock.
- `LIB_AUDIT` : append audit entries and produce run manifest summaries.
- `POSTPAY` : accepts payment request, checks idempotency, posts ledger entry, updates current balance.
- `REVERSE` : validates a reversal, ensures original exists, writes reversal entry, updates current balance.
- `BALINQ` : reads indexed balance and recent postings (N entries) for diagnostics.
- `RECONCILE` : nightly job: totals by date/channel/currency; cross-check vs input totals; produces exception report.

Sample COBOL program skeletons (high-level)

`POSTPAY.cb1` (shape only):

```
IDENTIFICATION DIVISION.
PROGRAM-ID. POSTPAY.

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
    SELECT PAYMENT-IN ASSIGN TO ...
    SELECT LEDGER-OUT  ASSIGN TO ...
    SELECT BAL-IDX     ASSIGN TO ... ORGANIZATION IS INDEXED ...

DATA DIVISION.
FILE SECTION.
FD PAYMENT-IN.
    COPY PAYMENT-REQ.
FD LEDGER-OUT.
    COPY LEDGER-REC.

WORKING-STORAGE SECTION.
01 WS-STATUS                PIC X(2).
01 WS-IDEMPOTENT-HIT        PIC X VALUE 'N'.
01 WS-ERROR-CODE            PIC X(10).

PROCEDURE DIVISION.
MAIN.
    PERFORM INIT
    PERFORM UNTIL EOF
        READ PAYMENT-IN AT END SET EOF TO TRUE
        NOT AT END
            PERFORM VALIDATE-REQ
            IF WS-ERROR-CODE NOT = SPACES
```

```

        PERFORM WRITE-REJECT
    ELSE
        PERFORM CHECK-IDEMPOTENCY
        IF WS-IDEMPOTENT-HIT = 'Y'
            PERFORM WRITE-AUDIT-IDEMPOTENT
        ELSE
            PERFORM APPLY-POSTING
            PERFORM WRITE-AUDIT-SUCCESS
        END-IF
    END-IF
END-READ
END-PERFORM
PERFORM FINALIZE
GOBACK.

```

RECONCILE.cb1 (shape only):

```

IDENTIFICATION DIVISION.
PROGRAM-ID. RECONCILE.

PROCEDURE DIVISION.
    PERFORM LOAD-INPUT-TOTALS
    PERFORM SCAN-LEDGER-JOURNAL
    PERFORM COMPARE-TOTALS
    PERFORM WRITE-EXCEPTION-REPORT
    GOBACK.

```

REST contract (façade)

- POST /v1/payments
- Request: { "traceId": "...", "accountId": "...", "amountCents": 12345, "currency": "USD", "postDate": "2026-02-25", "channel": "API", "merchantRef": "..." }
- Response: { "status": "OK|DUPLICATE|REJECTED", "internalTxnId": "...", "balanceAfter": ..., "auditRef": "..." }
- POST /v1/payments/{traceId}/reverse
- Request: { "reason": "..." }
- GET /v1/accounts/{accountId}/balance
- Response includes current balance + last N postings.

Implementation note: the façade can call COBOL via subprocess (MVP) and later evolve to a shared-module interface (compile with `-m`) if you want lower latency; both are supported by the compiler's build modes.

Test strategy

- Unit: validate idempotency logic, reversal rules, field validations (Python).
- Golden-master: run batches with deterministic synthetic inputs; snapshot ledger + totals outputs; diff on refactors (reuses Idea G harness).
- Invariants: “sum(postings) + openingBalance = closingBalance” per account/day; “reversal cancels original amount.”

Demo script (repeatable)

1. `docker compose up --build`
 2. Generate data: `python data/synthetic/gen_accounts.py` and `gen_payments.py`
 3. Load batch posting via REST: `scripts/load_demo.sh` (calls API; API writes request file; invokes `POSTPAY`)
 4. Replay same request set to prove idempotency (no double postings)
 5. Trigger reversal on a subset via REST
 6. Run reconcile: `scripts/run_reconcile.sh`
 7. Show metrics: postings/sec, duplicates avoided, reconcile pass rate, exception report count
-

Build plan for Idea I: COBOL auto-documentation + dependency and data I/O inventory

Folder structure

```
cobol-docgen/  
  analyzer/  
    src/  
      scan_programs.py  
      parse_copybooks.py  
      extract_calls.py  
      extract_fileio.py  
      graph_build.py  
      render_markdown.py  
      render_mermaid.py  
    fixtures/  
      expected_graphs/  
      expected_inventories/  
  sample_codebase/  
    cobol/  
      ACCTUPD.cb1  
      BILLRUN.cb1  
      CLAIMVAL.cb1  
    copybooks/  
      ACCT-REC.cpy  
      CLAIM-REC.cpy  
      COMMON-ERR.cpy  
  jobs/
```

```

    nightly_billing.job.yml      # JCL-like definition for dep mapping
out/
  docs/
    system_inventory.md
    call_graph.mmd
    copybook_graph.mmd
    fileio_inventory.csv
    impact_reports/
cli.py
README.md

```

Features (MVP scope 2-4 weeks)

- Call graph extraction: `CALL "XYZ"` / dynamic calls where detectable.
- Copybook graph: `COPY ABC.` edges.
- File I/O inventory: `SELECT`, `FD`, record names, file organization hints.
- Inventory outputs: Markdown summary + CSV for spreadsheets.
- Impact analysis: "If copybook field changes, which programs read/write it?" (best-effort based on symbol matching and FD usage).

This directly addresses modernization guidance that emphasizes managing application interdependencies and documentation during modernization. ⁹

Module boundaries

- `scan_programs`: discover `.cbl/.cob` files, normalize encoding, strip comments.
- `parse_copybooks`: resolve COPY paths, build inclusion tree.
- `extract_calls`: parse program IDs and CALL sites.
- `extract_fileio`: parse FILE-CONTROL/FILE SECTION patterns.
- `graph_build`: build directed graphs, compute transitive dependencies.
- `render_*`: Markdown + Mermaid output.

Sample COBOL skeletons (for the sample codebase)

`BILLRUN.cbl` ("reads CDR-like records, writes invoice file"):

```

IDENTIFICATION DIVISION.
PROGRAM-ID. BILLRUN.
ENVIRONMENT DIVISION.
FILE-CONTROL.
    SELECT CDR-IN ASSIGN TO ...
    SELECT INV-OUT ASSIGN TO ...
DATA DIVISION.
FILE SECTION.
FD CDR-IN.
    COPY CDR-REC.

```

```
FD INV-OUT.  
  COPY INV-REC.  
PROCEDURE DIVISION.  
  CALL "RATE" USING ...  
  CALL "WRITEINV" USING ...  
GOBACK.
```

ACCTUPD.cb1 (indexed file usage signal):

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. ACCTUPD.  
ENVIRONMENT DIVISION.  
FILE-CONTROL.  
  SELECT ACCT-IDX ASSIGN TO ... ORGANIZATION IS INDEXED ...  
DATA DIVISION.  
FILE SECTION.  
FD ACCT-IDX.  
  COPY ACCT-REC.  
PROCEDURE DIVISION.  
  READ ACCT-IDX KEY IS ...  
  REWRITE ACCT-REC  
GOBACK.
```

CLI contract

- `python cli.py scan sample_codebase/ --out out/docs/`
- `python cli.py graph --format mermaid`
- `python cli.py impact --copybook ACCT-REC.cpy --field BALANCE`

Test strategy

- Fixture-based tests: small COBOL snippets with known extracted edges.
- Golden outputs: expected Mermaid graphs compared in CI.
- Negative tests: ensure analyzer emits “unknown/unparsed” warnings for unsupported constructs instead of silently skipping.

Demo script

1. Run `scan` to generate inventory + graphs.
2. View `call_graph.mmd` and `copybook_graph.mmd`.
3. Simulate a change: add a field to `ACCT-REC.cpy`.
4. Run `impact` and show the impacted programs list.

Build plan for Idea G: Golden-master regression harness for COBOL batch refactors

Folder structure

```
cobol-gm-harness/  
  programs/  
    v1/  
      RATEBILL.cbl  
      copybooks/  
        CDR-REC.cpy  
        INV-REC.cpy  
    v2/  
      RATEBILL.cbl          # refactored or "optimized" version  
      copybooks/  
        CDR-REC.cpy  
        INV-REC.cpy  
  harness/  
    gm.py                   # CLI entry  
    runner.py               # compile + run  
    canonicalize.py         # fixed-width field-aware diff  
    diff.py  
    report.py  
  fixtures/  
    cases/  
      small.yml  
      edge.yml  
      large.yml  
    inputs/  
      cdr_small.dat  
      cdr_edge.dat  
  golden/  
    v1/  
      inv_small.dat  
      inv_edge.dat  
      totals.json  
  reports/  
  README.md
```

Data formats

Use a fixed-width CDR file with deliberate edge cases (boundary durations, missing fields, negative/overflow-like scenarios). Output invoice file is also fixed-width plus a totals JSON.

Example CDR schema:

CDR-REC (length 80)			
1-12	CUSTOMER_ID	X(12)	
13-26	CALL_START_TS	9(14)	YYYYMMDDhhmmss
27-32	DURATION_SEC	9(6)	
33-36	DEST_ZONE	X(4)	
37-46	PLAN_ID	X(10)	
47-55	USAGE_UNITS	9(9)	
56-80	FILLER	X(25)	

Sample COBOL skeletons (program under test)

RATEBILL.cb1 v1:

```

IDENTIFICATION DIVISION.
PROGRAM-ID. RATEBILL.

PROCEDURE DIVISION.
    PERFORM LOAD-RATE-TABLE
    PERFORM UNTIL EOF
        READ CDR-IN ...
        PERFORM APPLY-RATING
        PERFORM WRITE-INVOICE-REC
        PERFORM ACCUMULATE-TOTALS
    END-PERFORM
    PERFORM WRITE-TOTALS
GOBACK.

```

v2 introduces a bug (e.g., rounding or a boundary condition) for demo purposes.

CLI contract

- `python harness/gm.py bless --version v1 --case fixtures/cases/edge.yml`
- `python harness/gm.py run --version v2 --case fixtures/cases/edge.yml`
- `python harness/gm.py diff --baseline golden/v1 --candidate out/v2`
- `python harness/gm.py report --format md`

Test strategy

- Golden-master comparisons:
- Field-aware diff for fixed-width outputs using schema offsets (pinpoint which field changed, not just the line).
- Totals-level assertions (aggregate consistency).
- Deterministic execution:
- Seeded synthetic inputs.
- Stable ordering and stable timestamps in “test mode.”

- Optional extension (if time remains): integrate property-based input generation and feed any discovered counterexamples back into `fixtures/cases/`. This aligns with modernization guidance emphasizing hybrid functional/nonfunctional testing during coexistence. ⁹

Demo script

1. Compile and run v1; bless outputs to `golden/v1`.
2. Run v2 on the same fixtures.
3. Show diff report:
4. record index
5. field name (from schema)
6. old/new values
7. Introduce a “performance optimization” (e.g., caching rate table) and prove correctness remains unchanged via harness.

¹ ⁸ ¹⁶ ¹⁸ ²⁰ ²⁸ Mainframe workloads: Batch and online transaction processing

<https://www.ibm.com/docs/en/zos-basic-skills?topic=today-mainframe-workloads-batch-online-transaction-processing>

² ⁷ ¹² ¹⁴ State of the Mainframe in 2025 – BMC Software | Blogs

<https://www.bmc.com/blogs/state-of-mainframe/>

³ ²¹ ²⁷ Strangler Fig Pattern - Azure Architecture Center | Microsoft Learn

<https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig>

⁴ ²⁵ Patterns for a Gradual Modernization of IBM Mainframes and Midranges using Azure Logic Apps | Microsoft Community Hub

<https://techcommunity.microsoft.com/blog/integrationsonazureblog/patterns-for-a-gradual-modernization-of-ibm-mainframes-and-midranges-using-azure/3919207>

⁵ ⁹ ¹¹ ¹³ ¹⁵ ²⁹ Mainframe and midrange modernization - Azure Logic Apps | Microsoft Learn

<https://learn.microsoft.com/en-us/azure/logic-apps/mainframe-modernization-overview>

⁶ GnuCOBOL - GNU Project

<https://gnucobol.sourceforge.io/>

¹⁰ ¹⁹ ³¹ Bridging the Skills Gap with the Open Mainframe Project's COBOL Programming Course - Open Mainframe Project

<https://openmainframeproject.org/blog/bridging-the-skills-gap-with-the-open-mainframe-projects-cobol-programming-course/>

¹⁷ ²⁶ Processing work on z/OS: How the system starts and manages batch jobs

<https://www.ibm.com/docs/en/zos-basic-skills?topic=zc-processing-work-zos-how-system-starts-manages-batch-jobs>

²² ²⁴ ³⁰ Defining record lengths for VSAM files

<https://www.ibm.com/docs/en/cobol-zos/6.3.0?topic=records-defining-record-lengths-vsam-files>

²³ Transactions in CICS

<https://www.ibm.com/docs/en/cics-ts/6.x?topic=applications-transactions-in-cics>

³² GnuCOBOL Manual

<https://gnucobol.sourceforge.io/doc/gnucobol.html>